

guarantee an easy path to success, but it may help the team over the “hump of pain” of learning automation (*Agile Testing*, p. 266).

Implementing Automation Solutions in a Large Organization

Here's a story from Cirilo Wortel, an automation test consultant in the Netherlands, about how automation solutions evolved in a large banking organization.

Being an all-around hands-on tester, with a lot of field experience, I was anxious to start coaching the next generation of agile testers. The company I work for specializes in agile coaching and was running a full-fledged agile adoption program at one of the largest banks in the Netherlands. The transition went remarkably well; big open spaces with enormous Scrum boards marked the office spaces, and lively collaboration between most disciplines was already common practice.

Something that was not as successful as the general agile adoption was the integration of testers into the teams. Yes, there were testers in every team, and yes, they were attempting to collaborate. But most of them had little experience with agile development, and only a few had experience with test automation. Our goal was to bring automation to a higher level and get the testers more involved in the process.

I started out coaching three teams that already had started to use FitNesse in combination with Selenium. Both tools were well known to me, which was one of the reasons I was selected for the job.

The teams faced similar practical problems. Test automation was immature, test environments were unstable, and there was no version control on either tests or tools. There also was no configuration management, which resulted in many different combinations of tool and browser versions. There was no concept of test architecture and no focus on test maintainability. Figure 17-1 shows what happens if there is no collaboration when there isn't any focus.

I spent my time on practical solutions, helping out with technical problems and introducing more mature configurations and version control. I had to explain the fundamentals of test automation and teach all the required technical skills. We (the assigned coaches and I) collaborated to try to create more synergy among programmers, testers, and analysts.

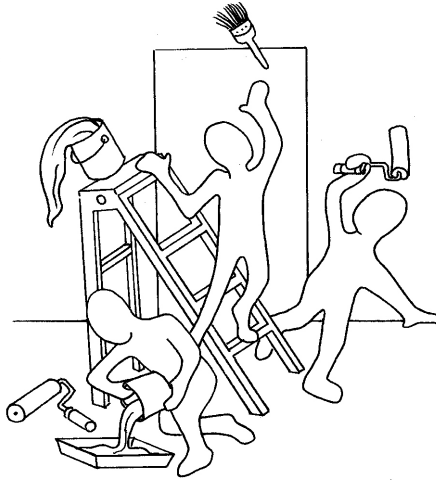


Figure 17-1 Working without collaboration

The teams I had started with became quite successful, and the news spread throughout the bank. Soon people in other departments started using the same configuration. This caused the number of teams I had to support to grow rapidly.

With this expansion came new obstacles. Lots of time was lost just doing installations and configurations, and there were many technical problems to solve. Besides more on-topic issues, like having to deal with annoyances or peculiarities of FitNesse and Selenium, there were also infrastructure or software problems, which were outside the scope of my activities. The more the new approach spread, the more obstacles appeared. It was soon obvious that the settled establishment was not so open to change.

I talked to managers who were promoting test automation and agile testing and encouraged the department of “Tool Support” to allow the tools we were using to be spread more easily. Convincing the “Test Automation Center of Excellence” that they would gain positive new options also took time. Often, bureaucracy and company “standards” were used as a barrier. The company’s use of commercial off-the-shelf tools was defended on the basis of their expensive licenses, although the tools served no purpose for the teams.

Many people fear change and will do anything to prevent it. The reasons they used against my proposed solutions ranged from funny to ridiculous and seldom made any sense. The company standard was simply not agile. I created the following rule set to convince my “opposition” that a change was required:

1. When working in an agile environment, automation is of **vital importance to keep up with the pace of the project**. Relying on only manual testing is not an option.
2. Automation must be possible within the multidisciplinary team; **no external expertise should be required** to create or maintain tests.
3. A test tool should be flexible, making it **easy to respond to change** in terms of environmental, data, and technical constraints.
4. Creating automated test cases should be relatively **simple and intuitive**.
5. Tests should be **readable and understandable** for anybody. Documenting and formatting tests in unambiguous and ubiquitous language supports this.
6. Tests should be **easily accessible and interchangeable** between team members and preferably also for other stakeholders. Anyone on the team should be capable of adding, viewing, and running tests.
7. Test execution should be **fast and reliable**, since fast feedback is the main objective.

With lots of persuasion and perseverance, not to mention teaming up with the right people, the popularity of the tool set grew, and most agile teams switched to using it in one way or another.

The tools got more popular, but the attention I could actually pay to coaching was limited. FitNesse is in many ways an excellent tool and allows users a lot of freedom. For many inexperienced testers, this can be too much freedom. With the limited time available to me, the entire endeavor got out of hand; it grew beyond my capabilities. It was time for more change!

In order to balance my activities, I had to spread knowledge in a more efficient way. To spread in-depth information, I started to organize courses. The courses would fit 10 to 15 people at a time, which was much more effective than passing information one on one.

We set up a wiki with basic knowledge on agile testing. It contained a substantial tool section, along with an installation guide, with work-arounds for most common installation issues. It had a FAQ section, filled with the most common problems collected in the past period, and contained actual methods that could be used in specific complex situations.

To actively build a community, I arranged knowledge sessions, and these monthly sessions attracted a substantial audience. Testers were invited from different successful teams to present their efforts, especially ones that used very different approaches to try to broaden people's views. These sessions were extremely valuable; people started exchanging ideas, and it seemed to me that more cohesion among the teams emerged. Figure 17-2 shows how collaborating and working together can move us forward.

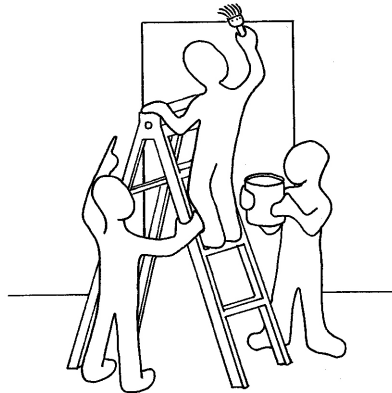


Figure 17-2 When we collaborate

Colleagues began to make FitNesse more appealing to a broader audience, which eventually resulted in a complete refactoring of the framework.

I collected more measures of progress. A management trainee started an experiment with two comparable teams to measure effectiveness, one using the agile approach and the other a more traditional method.

Unfortunately, just when things were finally on a roll, an organizational change meant my role was no longer funded and my assignment ended.

In retrospect, I have to conclude that the degree of success of my work was somewhat disappointing. The overall level of quality was below my personal standards, and some teams actually completely failed at automation. When considered from a broader perspective, however, the testers became more involved in test automation and in the agile process. I often heard the change had drastically increased their job satisfaction. Many teams still use the same approach and the configuration that was introduced, and it became the standard for agile projects within the bank.

What in the end is most important to me is the personal growth I experienced. A learning experience like this is invaluable.

Cirilo succeeded in helping teams within the banking organization implement a specific tool set that worked for their context. Organizational culture is difficult to overcome, but he had good strategies for helping teams succeed with automation. He set rules so that the approach would be consistent across teams. He and his colleagues refactored one of their open-source test frameworks, FitNesse, to better fit their needs. He found effective ways to measure progress, and he built a test automation community. Consider these ideas as your team implements your own test automation solutions.

MEETING NEW AUTOMATION CHALLENGES WITH THE WHOLE TEAM

If you need to fill a gap in your test automation, improve your automation ROI, or automate something that defies current solutions, get your whole team thinking of new experiments to try.

Lisa's Story

Our team was writing a new version of our API. Each time we tested a newly delivered story, we found missing requirements. There were also multiple failures, often with authorization issues and boundary conditions. We ended up rejecting the stories multiple times, which was costly.

We discussed this problem and realized that the stories were written for an audience of programmers who were familiar with old versions of the API.

However, recently hired programmers were coding the stories, and they weren't familiar with all the details of how the API should work.

We decided that the best way to express examples would be acceptance tests that would be automated as the story was coded. We needed a framework, so I paired with a developer to spike a specification by example (SBE)-style framework. In hindsight, we should have had two pairs, each trying a different approach, taking advantage of the benefits of set-based development. Still, I thought we came up with an interesting domain-specific language (DSL), related to the RSpec approach that our team was using for test-driven development (TDD).

Unfortunately, when we showed our scripts to the rest of the team, they were worried that our tests, which tested the API through the HTTP layer, had too much duplication with the functional RSpec tests written as part of the TDD process. I personally disagreed but felt I had to go with the team desires.

I still wanted to help guide coding at the acceptance or story test level, so I wrote tests in pseudo-code fashion, using a similar DSL to what we had tried for our first SBE effort. I was able to get a programmer to pair with me to automate my test cases in this modified DSL. We found three bugs in the process of automating them!

Once these were working, I specified tests in a similar fashion for a different API endpoint. The programmers used these tests as they worked on the stories, found them helpful, and made suggestions for improving them, along with additional test cases. The number of rejections for API stories went down, showing the value of using tests to illustrate requirements.

However, some of the programmers found that the learning curve for the modified DSL was extra overhead for them. Though it was more readable for testers, there was enough of a difference from their own RSpec tests that it felt like task switching. They felt it was slowing them down too much. Our team's culture was that the pair writing the code also automated tests at all levels. Switching DSLs seemed too expensive, so they wanted to use the same RSpec format for the SBE tests.

One downside to this approach was that the tests couldn't go through the HTTP layer, so some tests couldn't be automated. This approach also meant I couldn't participate in the test automation, but we needed an approach everyone was comfortable using.

I continued to specify test cases using a pseudo-code format, documenting them on the team wiki, and linked those tests to the story. The programmer pair then automated these tests using the same framework as for their unit tests. This was still a type of SBE approach, and it did provide value, although it felt clumsy to me. Also, the test cases that required going

through the HTTP layer couldn't be automated, which led to holes in the automated regression test suite. We have to do more manual checks to compensate, and this constitutes testing technical debt, slowing us down.

I continued this practice for the major endpoint stories, but it felt too heavyweight for smaller stories. I switched to listing test cases in the story itself, but I found that programmers sometimes missed them, and the delivered story got rejected. In my experience, if the programmers aren't including automating the acceptance tests as part of their development effort, they will often miss those requirements.

I found that after this series of experiments, the programmers thought of more test cases on their own as they coded using TDD. Guiding development with customer-facing tests is new to the company culture, and it will take more discussions and experiments to find the right approach.

In our daily efforts to deliver value to our customers, it can be hard to take time to try something new. However, that investment is worthwhile to improve our ability to build quality and value into our software products over time.

ACHIEVING TEAM CONSENSUS FOR AUTOMATION SOLUTIONS

Getting a team, especially a large team, to agree on an automation framework or other solution can be tough. However painful it may be, when you're faced with a tough problem, the whole team needs to discuss it and come up with experiments to try to find a solution.

Lisa's Story

For eight years, my previous team had been satisfied with our user interface (UI) automation framework. Our tests kept regression failures out of production, and though they could have been better designed, they were reasonable to maintain.

But we needed some new features in our UI to help prevent users from making mistakes that caused our team to lose valuable time doing production support. The programmers decided they should use Ajax to implement these features, but our test tool wouldn't work with the client-side code. We had our own rule to never release code that wasn't supported with automated regression tests, so although the coding solution was ready, we had to defer delivery until we addressed the test automation problem.

We researched some potential automation drivers and frameworks but needed a plan of action. I got the team together for a meeting to decide

what to do next. After we discussed the problem for a while, there was a long, painful silence. I was sitting there thinking, “Yeah, I tell other people how to solve automation problems, but I can’t even get my own team to talk about it.”

I gathered my courage and listed some of our potential options. We could hire an experienced coach to come in and tell us what UI test driver and framework to use and teach us how to use it. We could keep looking for drivers and frameworks on our own.

Finally, from behind a cube wall (see Figure 17-3), our system administrator (whom I had failed to invite to the meeting) piped up. “I think WebDriver will do what we need. I’ll do a spike.” We were all relieved. Finally, a plan!

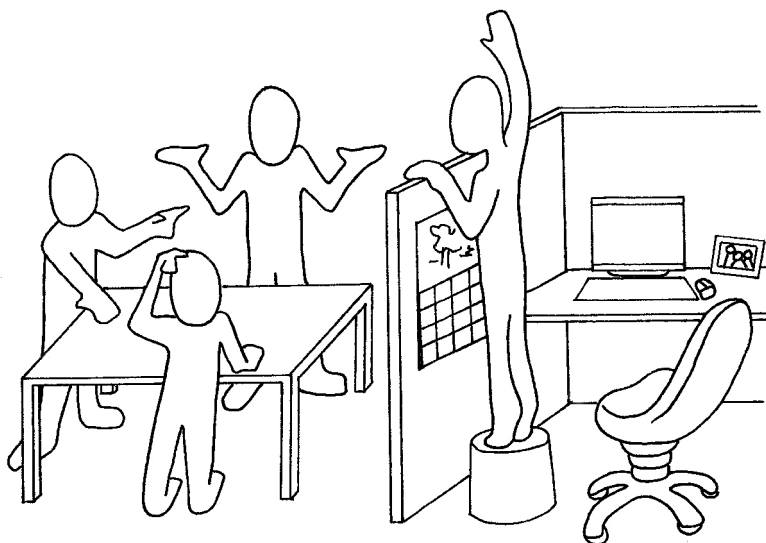


Figure 17-3 Invite the right people

The spike succeeded, and we moved on to “bake-offs” to determine what framework would work best for us with WebDriver. It was all a big investment, but the most important investment was that initial, painful meeting.

Even when you think your team has good regression test coverage in place, changing customer needs and new technology may require a new approach. Whether you’re taking your initial baby steps in meeting automation needs or considering a change to a previously successful approach, make sure you invite the right people to help address the

problem. The whole-team approach works when there's a diversity of skills and perspectives. And when all else fails, hold your meetings within earshot of the people you forgot to invite.



What's your team's biggest problem right now? Is it related to automation? Maybe you need a way to identify performance bottlenecks or holes in security. Pull interested people together. Brainstorm ideas. Do a proof of concept. Experiment with some bake-offs. Seek outside expertise if needed. Spend time learning, and, yes, this is often extremely hard to do! Get a like-minded teammate to help, and remember the agile tester value of courage.

Yes, it's hard to step back and think of ways to chip away at a difficult problem when your customers want features X, Y, and Z out the door. Always plan your workload so that you have time to make progress in areas that are crucial to your team's long-term success, such as the right automated tests.

HOW MUCH AUTOMATION IS ENOUGH?

In Chapter 14 of *Agile Testing*, we gave examples of testing activities that benefit from automation, as well as those that need human involvement or where automation is difficult. Over the years, we've enjoyed many new and improved test frameworks and drivers that help us use automation in more ways and with less pain. For example, we can write and manage our test code in the same integrated development environments (IDEs) and source code control systems as our production code.

As teams produce more production code and its accompanying test code, we face a new problem, especially with respect to automated regression checks. Continuous integration (CI) should provide quick feedback if any changes have introduced a regression failure. But as you add new regression tests each iteration, test suite build jobs take longer and longer. It's easy to lose that short feedback loop.

We can speed up our builds in various ways, perhaps by using virtual machines to run many test suites in parallel. Even so, we have to investigate every test failure. More tests mean more possibilities of failures that aren't a regression in the code but a problem with the test itself. More tests mean a larger test code base that requires refactoring and updating to stay current with production code.

Lisa's Story

Our team struggled with the CI being red for days on end. At least one pair was devoted 100% each day to getting the builds to green. This was a major roadblock to getting stories delivered, tested, and accepted. And real regression failures often weren't identified for hours or days. The biggest problems were with tests through the UI, which failed spuriously for various reasons. This was a constant topic of discussion at team retrospectives, but solutions were elusive.

The development manager created a shared document where team members could brainstorm ways to tackle the long-running and often-failing CI build. We were invited to share problems, concerns, questions, dilemmas, and realities, as well as potential solutions or improvements.

This was turned into a spreadsheet. For each idea, we listed benefits, disadvantages, risks, and notes about complexity. A small group of interested programmers and testers with a variety of skills met to discuss each idea. We rated each idea on benefits and complexity, using a point scale of 1 to 10. Then we subtracted the complexity score from the benefit score.

Using the results, we identified the "low-hanging fruit" and wrote stories to do these. Some were fairly simple; for example, we started rebooting the CI boxes nightly to help ensure that the builds always started with a clean environment and no stale or hung processes.

The browsers in the test environments that ran the end-to-end tests were subject to automatic browser version updates that often caused test failures. A second set of test environments was created, where the browser settings were changed to prevent automatic updates. That made it easier to know if an automatic browser update caused a test failure.

Another idea was to put together a short-running smoke test that would verify the most critical parts of the UI and take only a couple of minutes to run. The full suite of UI tests would still run, but the automated deploy to the test environment would be based on the success of the smoke test in addition to the unit and functional-level tests.

These high-benefit and relatively simple actions improved the stability of the graphical user interface (GUI) test suites. Newly finished stories are now deployed more regularly, evening out our workload and making it easier to do regular releases.

It is valuable to use automation to help verify every edge case and boundary condition during development. However, depending on the system under test (SUT) and the risks involved, it may not make sense to run every permutation of every test case in the CI. Not every test case run during coding a story may need to be added to the regression suite.

You might have a particular suite of selected integration tests that are slow to run, but it's enough to run them once a day or perhaps just before release. Depending on your situation, the same may apply to performance tests, UI tests, or any tests that may be inherently slow and can be run less often without too much risk.

COLLABORATIVE SOLUTIONS FOR CHOOSING TOOLS

When you are deciding what tools to choose, there are many considerations to evaluating any given one. In Chapter 14 of *Agile Testing*, we advised taking on one tool need at a time, identifying your requirements, and deciding what type of tool to choose or build that fits your needs and infrastructure. Since we wrote that, we've learned even more about good ways to choose the right test drivers and frameworks, as well as other tools, for your team. As we mentioned in Chapter 16, "Test Automation Design Patterns and Approaches," your team should first decide how you want your automated tests to look, and then find tools to support that. If you choose the tool before you know how you want to express your tests, you've automatically eliminated many of your options.

One of the most important characteristics of a test framework is whether it fosters collaboration among the different roles on a team. In Chapter 16, "Test Automation Design Patterns and Approaches," we talked about the test method that glues the tests to the production code and the criticality of collaboration between programmers and testers. It's essential to use tools that enable and enhance this collaboration. Table 17-1 shows some of the differences between what we think of as collaborative tools and those that are noncollaborative.

The one negative comment we have heard from some people is that when programmers spend time working on tests, that's time they are not spending writing production code. However, programmers who work in the way we have described have told us that it helps them write better, more testable code, with fewer defects.

SCALING AUTOMATION TO LARGE ORGANIZATIONS

In large organizations, it is typical to have multiple teams working on the same product line, often five, six, or more. This has repercussions for

Table 17-1 Collaborative versus Noncollaborative Tools

Collaborative Tools	Noncollaborative Tools
Enable testers/business to define tests	Tests are usually through the UI
Test code can be in a common programming language	Programmers aren't usually willing to help
Programmers can run tests as they code	Tests are implemented after the code is written
Testers can ask programmers for help	Testers create and implement all tests
Make tests visible so team members can discuss testability before coding	

how we think about many aspects of testing the product. In this section we will concentrate on ways to scale automation for large, multiteam products.

Common test standards, including naming conventions, become even more critical when several teams work on a shared code base. Different teams may be touching different pieces of the production code and need to be comfortable changing tests and the test code behind them.

There also needs to be consistency in tool selection. If you are part of a small colocated team working on one product, selecting the tools that are best suited for your needs is relatively straightforward. When you need to consider other teams, and perhaps a support and maintenance team, there's a compelling need for common tools.

One way to accomplish this is to define classes of tools that are acceptable for different needs. For example, if you are on a team that works on Java code on product X, you may have a standard calling for Selenium for your UI tests, a choice of either Cucumber or FitNesse for your API (service)-level tests, and JUnit for your unit tests. This gives the team some autonomy but allows consistency for the maintenance team and other teams working on the same product.

Cirilo Wortel's sidebar earlier in this chapter illustrates how a common set of tools, guidelines, and community building promote successful test automation across a large organization. Let's look at another example.

Transition Challenges: Agile Test Automation

Geoff Meyer, a test architect with Dell Inc., explains how his large development organization found a more suitable approach to test automation.

As the Dell Enterprise Solutions Group (ESG) transitioned to agile-based development, one of our projects in particular illustrated some of the test challenges inherent in such a transition. The project team was staffed with top-notch programmers and testers, one of our top development managers, and probably one of the best and highly engaged product owners in the organization. As with most of our projects, this one also had an extensive hardware test matrix. The team recognized that test automation was a critical success factor and took steps to ensure that as features were developed, an automated regression suite was in place to guarantee working software across the broad compatibility matrix.

Unfortunately, one of the things that eventually slowed the project's ability to add features was the test automation philosophy. The test manager and test members responsible for the effort brought with them many years of experience, most of which were founded in black-box testing and UI automation on waterfall-based projects. Although the testers on the team quickly transitioned to the collaborative, Scrum-based, two-week sprint cycles, their test approach remained firmly founded in a UI-based test methodology.

As the launch date of the first release approached, the team found itself in catch-up mode to update the UI-based automation scripts to accommodate the late-breaking changes to the UI in the final sprints. As a result, the team had to undertake unexpected manual testing efforts while it simultaneously updated the automated UI tests. On the plus side, the UI-based automation test suite enabled an automated regression suite. However, because the functioning regression suite was two to three sprints behind the completion of the user stories, the project incurred a much longer manual test back end than had been originally planned.

We revisited the test automation philosophy for the project in the follow-up release. Because the application used a model-view-controller (MVC) architecture, we proposed a change in the test automation approach from UI-based to the SOAP (Simple Object Access Protocol) service level. Because of the relative stability of the SOAP interfaces versus the UI interface, we were confident that the test members of the Scrum team could develop in-sprint test automation at the service level, thus minimizing the need for additional testing on the

back end of the schedule. The resistance to this change in approach was captured quite succinctly by one of the Scrum team test members: "If we do web services automation and UI automation, then we're doing double the effort."

I was unable to convince the team before the start of the next release that what was actually being proposed was a trade-off. Instead of doing UI automation within the sprint, the team would focus their in-sprint test efforts on automation at the service level and manual testing of the UI. The second release had similar results, but the problem was worsening. Because the automation backlog was growing, an increased number of testers had to be assigned to complete manual testing. Prior to this release, the ratio of programmers to testers was 3:1. During this release of the project, that ratio began to approach 1:1.

As the team prepared for the third release, I took another run at altering the team's approach to test automation. We assigned one of our most experienced service-level automation testers to dig into the application with the goal of demonstrating how service-level automation could be used to help the team keep the test automation on pace with new development. Unfortunately, after two full releases, the inattention by the team on the testability and automatability aspects below the UI had led to more business logic and error handling being built into the UI layer instead of below the services interface. After detailed analysis, the business team decided to refactor the business logic and error-handling application over the course of the next few releases to further enable a services-based test automation approach.

There were several lessons that we've taken away from this experience. First, a test strategy based predominantly on UI automation is likely to lead to unplanned time and effort to harden the product late in the cycle. Also, even when members of the Scrum teams have a shared understanding of the importance of test automation, determining the appropriate types of automation (UI or service level) within the application and the right time to employ them is equally important. Last, the recognition that the application architecture can be either an enabler or an inhibitor to a project's test automation strategy is an important lesson to apply throughout all stages of a product's life cycle.

Insufficient test automation hurts your product and its users, and the risk of regression failures escaping to production is higher. If your team has a policy to release only those features supported by automated regression tests, implementation of new features that could make the product more usable or valuable may be held up by an inability to

automate tests. The product's architecture may need refactoring to support the improved ability to automate.

OTHER AUTOMATION CONSIDERATIONS

So far we've focused on functional test automation. Automation is also required for most Quadrant 4 tests, the technology-facing tests that critique the product, such as performance, load, scalability, capacity, reliability, and installability testing. Tooling is also needed for monitoring and checking memory management. Then there's all the tooling needed for CI, configuration management, version control, the deployment pipeline, and data management. Check Part VI, "Test Automation," and the "Tools" sections of the bibliography for sources that will guide you in those areas.

Automated testing is a series of checks, but we use it first to help guide development with customer-facing examples turned into tests. We need it for living documentation to show how our system behaves. We need it to quickly inform us of regression failures. We need it to keep technical debt to a manageable level and to ensure that we have enough time for all the valuable testing activities that help deliver high-quality software.

Even if your team has solid automated test suites in place, be prepared for continual challenges as your product and the technology you use evolve. Be alert for innovations that can help with tough automation problems. For example, Lisa's team recently discovered a production issue that was missed in spite of extensive unit test coverage. To find a better regression test approach for this, one of the developers experimented with generative testing. In a generative test, you define expected properties for expected outputs instead of hard-coded values, and the test runner generates randomized data for checking. It even "shrinks" the test failures to the minimal failing case automatically (Kemerling, 2014). The new tests found reproducible failures so the code could be fixed. These didn't replace the unit tests, but they provided additional confidence.

Use the whole-team approach, always work on your biggest problem, and be patient. Your automation needn't be perfect to be valuable, and you should continually make it better.

SUMMARY

There is no one right test automation solution. Be patient; involve the whole team in experiments to find good solutions for your team. Use the pyramid models described in Chapter 15, “Pyramids of Automation,” to help. In this chapter, we looked at some of the most challenging steps in implementing valuable test automation. Some key points:

- Teams new to agile benefit from having an experienced practitioner who can help select solutions, find ways to measure progress, set standards across teams, and build community to share experiences.
- Involve the whole team in selecting automation solutions. It can take courage to get consensus on decisions about how to proceed. Focus on trying experiments, knowing there is no one “right” solution.
- Be judicious in what you include in automated test suites, and keep builds fast for timely feedback.
- Try to find automation frameworks that promote collaboration among testers, coders, and practitioners with other specialties on the team.
- Take time to learn together and grow the technical solutions; that is often where much of the value is created.
- Push automation to the lowest level that makes sense. Refactor code architecture if needed to allow for cost-effective test automation.

This page intentionally left blank

WHAT IS YOUR CONTEXT?

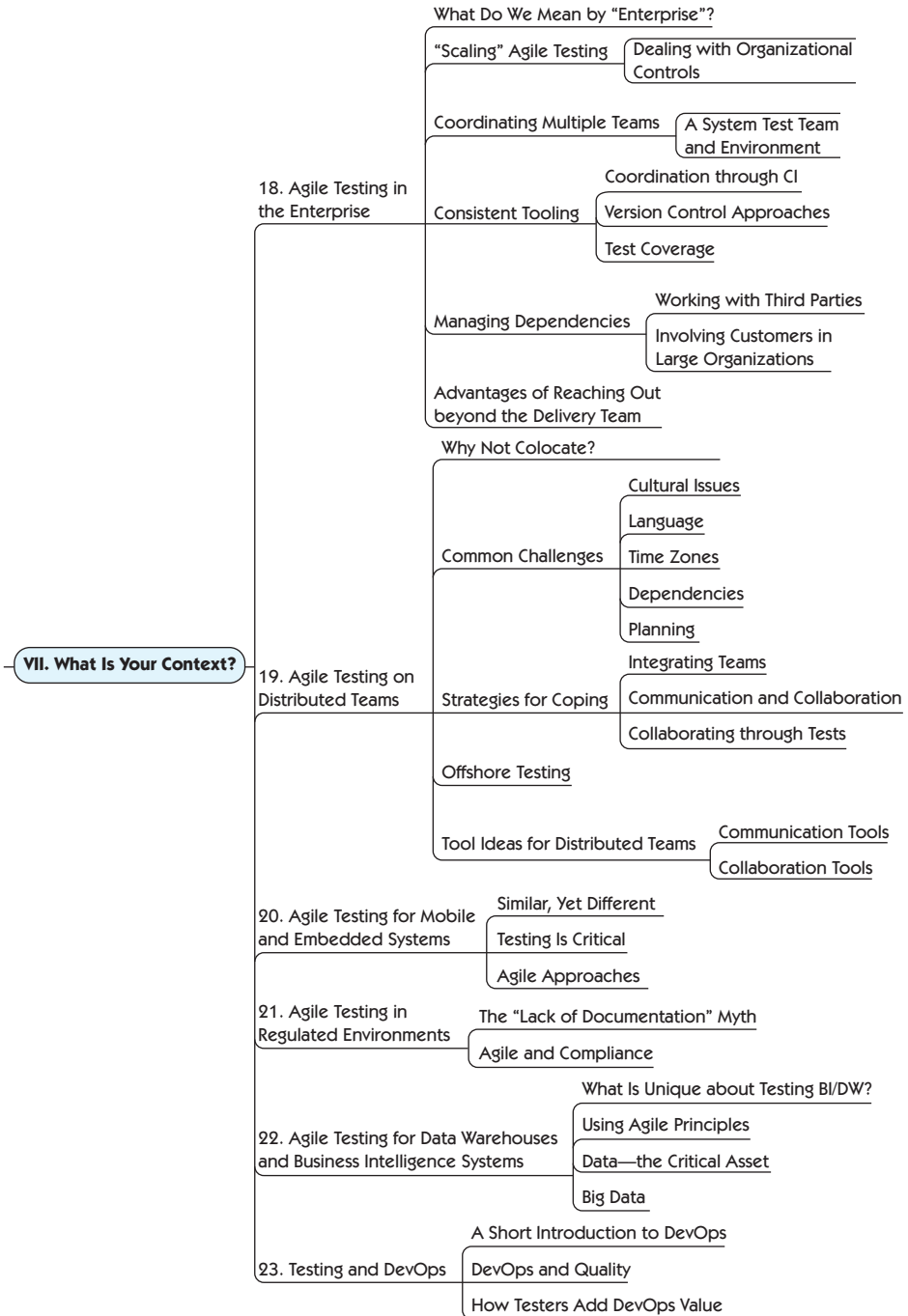
Context counts. In this section we will present a set of specific challenges that may not apply to everyone. We've gathered many stories from expert practitioners working in these diverse areas to help you.

Agile development has spread to large organizations with enterprise-wide solutions. We start this part of the book by looking at the particular challenges of “scaling up” agile testing. Chapter 18 is centered on the story of how one such organization transitioned its testing to agile.

Organizations large and small find themselves increasingly spread among different locations, perhaps around the whole planet. Our next chapter is, therefore, about testing on distributed teams. We will share strategies to cope with language, cultural, and technological barriers to keep everyone engaged in testing before, during, and after coding.

Other areas that have expanded dramatically over the past few years are mobile apps and embedded systems. These applications have high risks—both for safety and for success—in a fast-changing, opinionated market. They also pose unique testing challenges. Chapter 20 will look at what you need to test and how to fit all that testing into short release cycles using a whole-team approach.

Another domain that has puzzled teams that want to embrace agile is regulated software. Many teams feel that compliance and documentation constraints are mutually exclusive with agile development. We will share some success stories in Chapter 21 that show how well agile can meet the demands of regulatory agencies. Testers have unique opportunities to contribute their domain expertise and collaboration skills to help build products that delight customers and satisfy regulators.



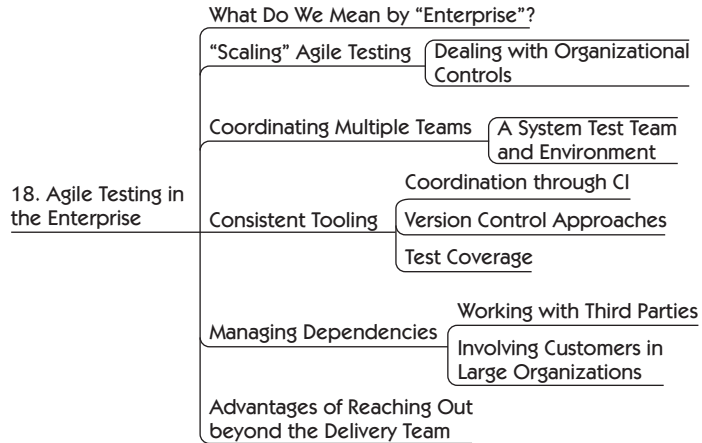
Testing data warehouses and business intelligence systems in agile environments also benefits from short feedback cycles and an incremental approach. In Chapter 22, we will share ways to expose data quality issues early. We will also discuss specialized technical skills and business domain knowledge needed to verify the data used for business decisions.

The term *DevOps* gained popularity starting in 2009, but practitioners with operations and development backgrounds have collaborated with testers and other team members for many years. In our chapter on testing and DevOps, we will show how testing and DevOps work together.

- **Chapter 18**, “Agile Testing in the Enterprise”
- **Chapter 19**, “Agile Testing on Distributed Teams”
- **Chapter 20**, “Agile Testing for Mobile and Embedded Systems”
- **Chapter 21**, “Agile Testing in Regulated Environments”
- **Chapter 22**, “Agile Testing for Data Warehouses and Business Intelligence Systems”
- **Chapter 23**, “Testing and DevOps”

This page intentionally left blank

AGILE TESTING IN THE ENTERPRISE



We hear a lot of talk about scaling agile. From the early days of agile, some experts have insisted that agile principles and values are only for small, colocated teams. Some practitioners believed that large enterprise organizations should not even try agile development. However, many have attempted an agile transition, and we’ve heard lots of success stories. The challenge is not so much scaling agile as trying to maintain the principles of agility in the context of organizations that mandate strict discipline and practices on their teams. This chapter is not meant to tell you to how to scale agile in large organizations. For that, we suggest you look in the bibliography for Part VII, “What Is Your Context?,” for books on that subject. Our focus is on good approaches for agile testing in enterprise companies.

WHAT DO WE MEAN BY “ENTERPRISE”?

First, we don’t mean the *Starship Enterprise*, although we often feel we are exploring strange new worlds and civilizations. We use the term *enterprise* to mean large, all-encompassing, crossing business units or

geographies, the whole of your business operations. “Enterprise” applies to more than large businesses. Universities, nonprofits, and governments are examples of huge organizations that need enterprise-wide software to meet a variety of needs. These organizations may be distributed, but not necessarily. Chapter 19 is about testing in distributed teams, so in this chapter, we’ll concentrate on large application systems, which usually mean large organizations.

Regardless of organization size, the focus should be on the customer. In 2009, Mark P. McDonald gave a bit of advice that we believe holds true for any company implementing agile and lean principles (McDonald, 2009):

Put the customer at the center of the company, because it is the one thing that you all share. That sounds trite but it’s true. Look to reduce barriers or impediments to internal operations that reduce customer service or make it harder for the customer to engage all parts of the company. After all as an enterprise your economic model rests on the ability to engage and serve the customer across your entire operations.

“SCALING” AGILE TESTING

In his keynote at Agile Testing Days 2013 (Hassa, 2013), Christian Hassa talked about scaling test-driven development (TDD) to the enterprise. His description sounded suspiciously similar to what is done in many agile teams. He recommended that teams set a desired goal, figure out the stakeholders, define a desired behavior change, determine deliverables, write a failing acceptance test, write the code to make that test pass using TDD, and so on. This sounds like the “Discuss—Distill—Develop—Demo” acceptance-test-driven development (ATDD) cycle as described by Elisabeth Hendrickson (Hendrickson, 2008).

Christian recommended that enterprises measure the impact of the deployed system and refine the deliverable accordingly. You can continually refine your strategy based on what behavior change actually happens, which elevates your testing. His conclusion was that scaling isn’t about how to do more work with more people. Rather, it pays to test goals and impacts as early and as often as possible, in any size organization.

Christian applies Tom Gilb’s planning language, Planguage (Gilb, 2013), for defining goals. In the process of defining goals, you select a scale: decide what to measure. Next, you choose a metric: how you will measure. Set levels by first benchmarking the current situation, setting constraints such as the minimum acceptable result, and defining the target, or desired result. Impact mapping, story mapping, and incremental and iterative delivery can all help you set goals and achieve the desired business value. Chapter 9, “Are We Building the Right Thing?” explained impact mapping and story mapping in more detail. The same techniques work in companies of all sizes, but as companies grow, more diligence is needed in setting goals and assessing impacts frequently.

Learning: Align the Target Setting across the Organization

Eveliina Vuolli, *operational development manager at Nokia (Networks Business), shares lessons she learned when trying to introduce automation in a large enterprise system.*

The whole organization needs to understand the benefit of doing test automation. For example, product management should be involved to ensure that they don’t see test automation only as a “funny R&D practice.” Instead, there needs to be a discussion of how test automation benefits product development; for example, automation improves the product quality in general and makes it easier to make last-minute changes in the software. In my experience, it is good to have example ROI calculations. They reinforce the message, especially when discussing test automation with the business stakeholders.

It is also good to have a common target setting for test automation throughout the whole organization. This is one way to create a common understanding of what the expected level of automation at different test levels is. When we did the target setting the first time, we used the same test automation percent coverage level as a target for all the teams. However, because the teams were in very different situations, the target number for one team was very challenging, whereas others had already achieved it. We learned that it is better to have the target setting done from the bottom up. You can still have a defined minimum level for all the teams, but above that, the exact target numbers may vary. In one target setting, the goals for different areas in unit test coverage varied between 60% and 95% and for the acceptance tests between 50% and 100% for a certain time period.

During the test automation (TA) target-setting discussions, the organization as a whole sometimes missed a common understanding of the testing-related terminology, which led to misunderstandings when we talked about TA status in different areas. We learned to start the target setting by defining the nomenclature. I have found it useful even to have an official definition for counting the test automation ratio. In principle it's a very simple formula: automated test cases/all test cases = TA ratio. With legacy software you need to agree on the formula and take old releases and their test cases into account in the calculation.

We found it essential to have a follow-up for the topic so that people realized it was important. If no one is asking about the topic after the "first rush," it loses immediacy, so we do the follow-up monthly. We have one centralized dashboard to show the status online.

We hear comments such as, "But the TA ratio actually doesn't tell much about the quality or coverage of the testing, even if it's 100%." Teams often need help to really consider how to increase the coverage of their TA cases or to optimize the run time of the automation rounds, so that they can move to the next level. Instead of just watching the number, they will also start to realize how to benefit from the test automation. One example is to have the computers do the "dummy work" so people can concentrate on more challenging testing areas or have a good safety net to enable refactoring and fast changes.

Dealing with Organizational Controls

Large organizations tend to have controls in place to ensure consistency. They develop over the years as the company grows and tries to solve specific problems. Many organizations are reluctant to change or drop these controls because that is what has worked in the past for them. One example would be enforcement of quality models such as CMMI (CMMI Institute, 2014) or internal audit requirements. Another example would be the way functional silos work together and have signed handoffs. In this chapter, we want to take you back to basics and have you think about how to apply the agile values and principles—even if you are dealing with enterprise solutions. We will talk more about regulatory restrictions in Chapter 21, "Agile Testing in Regulated Environments."

The history and culture of the organization are the most difficult to overcome. We have found that to be successful, teams often have to

prove themselves and be able to articulate what benefits agile can offer. We also need to be able to articulate to management how teams can still give them confidence in their ability to deliver without the sign-offs, without the constant reporting, and without the existing measures. Management has learned over many years how to make decisions based on those reports, and teams need to be able to give them a realistic alternative to the old reporting structure. It is not enough to say, “Trust us.” They need feedback loops, too.

Janet has found that using simple but visible tools such as the test matrix in Figure 7-6 in Chapter 7, “Levels of Precision for Planning,” can boost the confidence level of management that testing is being considered and tracked.

The Dell Journey—Part 1

Geoff Meyer, a test architect at Dell, shares the agile journey he has experienced at Dell. His story is similar to that of many organizations we have seen. Here’s his story of Dell’s initial steps toward agility. It is broken up into five parts, and we’ll examine each step along the way.

The Challenge

In most large enterprises, developing and testing software is not always a nimble process. Such was the case with Dell’s Enterprise Solutions Group (ESG) in 2008. Functional silos and waterfall development processes had created an entrenched and antagonistic “development versus test” culture. This lack of cohesion was exacerbated by teams being physically located on different floors and sometimes different time zones. Moreover, the test practices that had been optimized over years of waterfall practices were based on the black-box approach, and test automation skill sets were limited to user interface (UI) automation tools.

The Context of Agile @ Dell in 2009

- The software projects encompassed Embedded Systems and Server Systems Management products.
- Functional silos had created an entrenched, antagonistic “development versus test” culture.

- Development and testing were physically located on different floors.
- The large hardware compatibility testing matrix was based on supporting three generations of servers.
- Testers conducted only black-box testing.
- Test automation skill sets were limited to UI automation tools.

The Solution

Enter Brian Plunkett and Dave Spott, two software development executives, both of whom had come to Dell with prior agile experience. It was then that our agile journey began, largely because of the full backing and support of these two agile proponents.

Dell ESG first adopted agile practices in mid-2008. We started on a small scale using a pilot project, with a single team consisting only of programmers. Within the first six months, the results of the pilot project were sufficient to demonstrate to managers across the organization (especially those in the Test organization) that this new approach to software development might just have some merit.

Because the Test team was hosted in a separate functional organization, a good amount of lobbying on the part of Dave and Brian was required to bring them into the agile fold. At first, it was a tough sell. How could we suggest such drastic changes to program management, project staffing, and consequently the testing itself? Initially, there was resistance, but once Test had experienced being an integral member of the iterative development process, the Test organization became one of the biggest proponents of and champions for agile.

As you can see from Geoff's story so far, Dell had the same problem many enterprise organizations do in adopting agile principles and practices. The testing organization resisted the idea of integrating with development. However, they were fortunate to have two executives on the management team to help sell the idea. If management has decreed that software teams "must" go agile, find someone on the management team to be a sponsor to stand behind the changes that need to be made. Recommend a pilot project first to work out the kinks in the process. Talk to the management team about what changes need to be made throughout the organization to be successful. Be willing to make compromises such as reporting defects, but have regular retrospectives with management and talk about benefits and trade-offs. Be aware that some

groups have understandable fears about making fundamental changes to the development and test process.

Let’s see what Dell did next.

Implementation: Bumps in the Road—Dell, Part 2

In the first full year of the agile adoption, it became evident that the traditional testing techniques of black-box testing, UI-based test automation, and the need to log all defects were not a good fit with the collaborative, time-boxed guidelines of Scrum. Within the Scrum team, the testers were developing and executing manual tests against the UI of each user story. The only automation that was being developed was test scripts against the UI. To top it off, a traditional performance indicator of the Test organization was “number of defects found”; therefore, test Scrum team members logged each and every defect. As you can imagine, the result was test tasks for a user story that did not complete within the sprint boundary, which led to carryover user stories. Moreover, test automation was unable to keep up with the subsequent changes to the UI in follow-on sprints. Testers spent more time on maintaining test cases than on developing and executing new ones. Finally, the entire Scrum team became burdened by the administrative overhead of processing defects.

And these were just at each Scrum team level. At the organization level, test automation was already recognized as a critical success factor. However, the directive to “go forth and automate” only compounded the team-level problems since different UI automation tools, such as Rational Functional Tester (RFT), Ranorex, and Telerik, were employed due to the differences in the product architectures.

As you can see from Geoff’s story, not all problems are people problems. Because of the short iterations and the need for quick feedback, many underlying issues such as architecture or tooling become very visible. Large organizations tend to move more slowly because of their structure and approval processes, so changes come slowly, and with difficulty. Large organizations often have the need for additional roles such as test architect. There are many moving parts to an enterprise solution, and no one team can know it all.

Geoff Meyer continues his story with Dell and explains how they solved some of the testing issues with their agile adoption.

Agile Evolution—Dell, Part 3

I came into the Test organization in 2010 as the test architect; my objectives included evolving the agile test strategy to support large-scale projects and improving the test automation capabilities. An essential read for these challenges was *Agile Testing: A Practical Guide for Testers and Agile Teams* by Lisa Crispin and Janet Gregory. The automation pyramid and agile testing quadrants became foundations for the next step in the evolution of our Agile @ Dell test strategy.

One of the first things we did was to analyze the state of testability and automatability of our application architectures below the user interface, such as the command-line interface (CLI) or web services SOAP or RESTful layers. Next, we started training our test engineers in these new skill sets. In the case of projects where we were onboarding new hires, we hired to these skills, as well as agile experience.

We then, purposely, curtailed any UI automation from within the Scrum sprints. We also removed the requirement to log all defects and instead encouraged teams to collaborate in real time to root-cause, resolve, and verify them. Finally, we directed our Scrum testers to “automate first,” at the service level, and worked with the product owner and development teams to incorporate test automation into the user story acceptance criteria.

The latter was a lofty goal, unimaginable to many in the Test organization at that time. However, over the course of the next two years, the Scrum testers became very efficient at test automation development and execution at the CLI and web services interfaces within the sprint. Test automation was able to keep pace with user story development, resulting in active automated regression and build verification suites running for each project. Figure 18-1 shows coding and testing activities within the two-week sprint cycle.

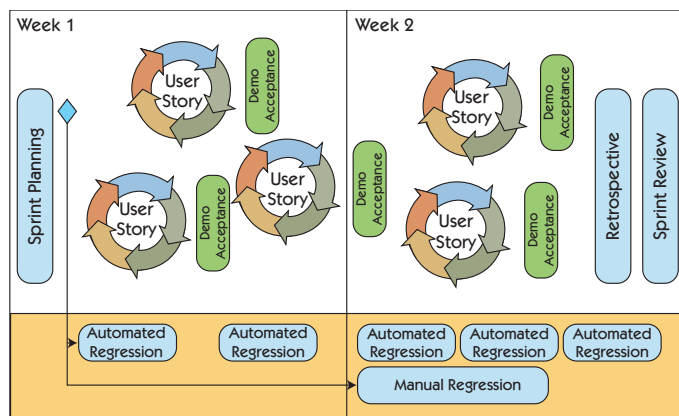


Figure 18-1 Two-week sprint life cycle example

We have noticed that in large organizations, where command-and-control hierarchies exist, it's harder for teams to self-organize and adapt than in smaller organizations with less structure. In Geoff's story, we see that a certain amount of direction was needed to get the teams to accept the new way of working. Over time, however, the new practices became the status quo.

COORDINATING MULTIPLE TEAMS



Testing in a large enterprise means thinking about many moving parts. Often, changing one component or one application has repercussions all the way through the system. More complexity is added when some teams are practicing agile, while others are working in a phased and gated traditional process. Agile transitions, which usually require big changes in company culture, are especially painful for large corporations. There are some unique challenges that teams in smaller companies do not face.

There are several possible ways you might approach multiple teams working on the same product. In Part III, “Planning—So You Don’t Forget the Big Picture,” we talked about levels of precision and test planning at each level for the product. We’ll use some of those concepts for testing in the enterprise.

Let’s use a simple example. Figure 18-2 shows a context diagram for a school system based on the assumption that the school system owns the buses. The new functionality that is needed is to automate bus scheduling so that parents are notified if there is a problem with delivery of their children. The scenario we will use as an example is “A bus breaks down, a new bus is dispatched to transport the children, and parents are notified the children will be late.”

The priority features we need are

- Ability to locate buses if they have problems
- Ability to contact a substitute driver and assign a substitute bus
- Ability to notify parents with text, phone, or email
- Ability to notify school administration of issues with bus maintenance and late delivery of children